

Rapport de projet informatique

Cohérence des Politiciens

Projet réalisé du 1er Septembre 2025 au 5 Janvier 2026

Membres du groupe

TEDJIOGNY Fabrice (42014524) et ONDZE Elroy (43012756)

<https://github.com/FabrTED/coherence-deputes>

MembreMembreMembreMembre

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude envers nos professeurs, qui nous ont accompagnés et guidés tout au long de ce projet. Ils ont également fait preuve de compréhension et de flexibilité face aux difficultés rencontrées, tant individuelles que collectives. Leur soutien et leur capacité d'adaptation ont été des éléments déterminants dans la réussite de ce travail.

Une mention particulière doit également être adressée aux différents outils d'intelligence artificielle utilisés, notamment ChatGPT d'OpenAI, qui se sont révélés être des aides pédagogiques précieuses tout au long du projet. Ces outils nous ont permis de mieux comprendre certains concepts complexes et de surmonter plusieurs difficultés liées au développement et au raisonnement informatique. Leur apport a contribué à rendre notre apprentissage plus efficace et plus structuré.

Enfin, nous remercions toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet. Leur aide et leur soutien ont été d'une grande valeur pour nous.

Table des matières

1	Introduction	4
2	Environnement de travail	4
3	Utilisation de l'intelligence artificielle	5
3.1	Aide apportée par l'IA dans la réalisation du projet	5
3.2	Fonctionnement général des outils d'IA utilisés	6
4	Description du projet et objectifs	6
4.1	Sous-Fonctions	6
4.1.1	Fonctions utilitaires et techniques	8
4.2	Programme principal (main)	9
5	Bibliothèques, Outils et technologies	10
6	Travail réalisé	12
7	Difficultés rencontrées	13
8	Bilan	13
8.1	Conclusion	13
8.2	Perspectives	14
9	Bibliographie	15
10	Webographie	16
11	Annexes	17
A	Cahier des charges	17
B	Manuel utilisateur	18
C	Exemple d'exécution du projet	19

1 Introduction

Dans le cadre de mon projet de **Deuxième Année (L2)**, j’ai développé un programme en langage **C** permettant d’analyser la **cohérence** entre les positions affichées par des députés et leurs **votes réels** sur plusieurs scrutins. L’objectif du projet est de mettre en pratique mes compétences en programmation C (structures, mémoire dynamique, fichiers, traitement de données), tout en proposant une application concrète liée à la **transparence politique** et à l’exploitation de données. .

Le programme repose sur un fichier **CSV** contenant, pour chaque député, un identifiant unique (**mp_id**), son nom, son parti, deux axes idéologiques déclarés (**axe_economie** et **axe_societal**), une catégorie de patrimoine (**patrimoine_classe**) et une suite de votes codés sur huit thèmes (**V001 à V008**). Ces votes sont représentés par des lettres simples : **P** (Pour), **C** (Contre), **A** (Abstention) et **N** (Absent). Afin d’éviter des erreurs d’affichage liées à la console, le protocole impose l’usage de caractères **sans accents** et, surtout, l’identification des députés via **mp_id**, ce qui évite toute ambiguïté en cas de noms similaires.

À partir des votes, le programme calcule automatiquement des **axes “réels”** (issus du comportement de vote) en pondérant les scrutins selon leur thématique (économie ou société). Il en déduit ensuite un score de **cohérence (0–100)**, basé sur la distance entre les axes déclarés (programme) et les axes calculés (votes). L’utilisateur peut alors consulter la **liste des députés**, afficher un **profil complet**, comparer **deux députés**, et obtenir un classement **Top 5 des plus cohérents / moins cohérents**. Pour rendre l’analyse plus visuelle, le programme affiche également un **diagramme 2D** en ASCII positionnant, sur les mêmes axes, le point “programme” et le point “votes”.

Enfin, pour faciliter la traçabilité et l’exploitation des résultats, le programme génère automatiquement des fichiers de sortie lors des opérations importantes : un fichier pour chaque **profil complet**, un fichier pour chaque **comparaison**, et un fichier pour chaque **classement Top 5**. Ces exports incluent systématiquement le **protocole** et les résultats, afin que les analyses restent compréhensibles hors du programme.

2 Environnement de travail

La totalité du développement de ce programme d’analyse de la cohérence politique a été réalisée sur un système d’exploitation **Windows**, environnement largement répandu et avec lequel je suis particulièrement à l’aise pour le développement en langage C et l’exécution de programmes en mode console.

Le **langage de programmation C**, imposé dans le cadre des enseignements de L2 MIASHS, a été utilisé pour l’ensemble du projet. Ce choix m’a permis de consolider des notions fondamentales telles que la **gestion de la mémoire dynamique**, l’utilisation de **structures de données**, la **lecture et l’écriture de fichiers**, ainsi que la conception de programmes modulaires et structurés.

Pour l’écriture, la compilation et le débogage du code, j’ai utilisé l’environnement de développement intégré **Code : :Blocks**, spécialement conçu pour le développement en **C**. Cet IDE m’a permis de bénéficier de fonctionnalités essentielles telles que la

coloration syntaxique, l'auto-indentation, la compilation intégrée et l'affichage détaillé des erreurs, facilitant ainsi la correction des erreurs de syntaxe et de logique tout au long du développement.

Dans le cadre de l'assistance à la conception du programme, à la résolution de problèmes techniques et à la clarification de certains concepts algorithmiques, j'ai eu recours à des **outils d'intelligence artificielle**, notamment **ChatGPT (OpenAI)**, ainsi que, de manière ponctuelle, **Gemini** et **Mistral**. Ces outils m'ont servi de support pédagogique pour structurer le programme, améliorer sa lisibilité et identifier des solutions adaptées au niveau requis, sans automatiser directement la production finale du code.

La préparation des données d'entrée a été effectuée à l'aide de **LibreOffice Calc**, utilisé pour créer et organiser les fichiers **CSV** respectant le protocole défini (structure des colonnes, codage des votes, absence d'accents). Les fichiers de sortie générés par le programme (profils, comparaisons et classements) ont été vérifiés et relus à l'aide de **Bloc-notes**, afin de contrôler leur lisibilité et leur conformité au protocole.

Enfin, les phases de test ont été réalisées en exécutant le programme directement depuis **Code : :Blocks**, en utilisant différents fichiers CSV et scénarios d'utilisation (saisies valides, erreurs de saisie, comparaisons multiples, génération de fichiers). Cette démarche m'a permis de vérifier le bon fonctionnement de chaque fonctionnalité avant validation finale du projet.

3 Utilisation de l'intelligence artificielle

3.1 Aide apportée par l'IA dans la réalisation du projet

Dans le cadre de ce projet, des outils d'intelligence artificielle ont été utilisés comme **support d'aide à la réflexion, à la rédaction et au développement**, sans automatiser ni remplacer le travail de conception, d'implémentation et de validation réalisé par les auteurs.

L'IA a principalement été mobilisée pour :

- Clarifier certains concepts théoriques liés à l'analyse de la cohérence politique (axes idéologiques, distances entre positions, similarité des votes).
- Proposer des pistes de structuration du programme en langage C (organisation des structures de données, découpage fonctionnel, modularité).
- Aider à la compréhension et à la correction d'erreurs techniques (gestion de la mémoire dynamique, lecture et validation de fichiers CSV, robustesse des entrées utilisateur).
- Améliorer la rédaction du rapport, notamment par la reformulation d'explications techniques, la structuration des sections et la cohérence entre le code source et la documentation associée.

Les choix algorithmiques finaux, l'implémentation du programme, les tests ainsi que l'interprétation des résultats ont été effectués manuellement par les auteurs. L'intelligence artificielle a donc été utilisée comme un **assistant méthodologique et pédagogique**, et non comme un outil automatique de génération de solution.

3.2 Fonctionnement général des outils d'IA utilisés

Les outils d'intelligence artificielle utilisés dans ce projet, en particulier **ChatGPT (OpenAI)**, reposent sur des modèles de langage de grande taille (*Large Language Models* – LLM).

Ces modèles sont entraînés sur de vastes corpus de textes afin d'apprendre les structures du langage, les relations sémantiques et certains schémas logiques. Ils fonctionnent par prédiction probabiliste de la suite de mots la plus pertinente à partir d'un contexte donné, sans compréhension ni raisonnement au sens humain.

Dans le cadre de ce projet, aucune intelligence artificielle n'a été intégrée directement au programme en langage C (absence d'API, de bibliothèque externe ou d'exécution automatisée). Les outils d'IA ont été utilisés exclusivement **en amont du développement**, comme aides externes à la conception, à l'apprentissage et à la documentation.

Il convient également de souligner que les réponses fournies par ces outils ne garantissent pas leur exactitude. Les propositions générées ont donc systématiquement été analysées, adaptées, corrigées et validées par les auteurs avant toute intégration dans le code ou dans le rapport final.

4 Description du projet et objectifs

4.1 Sous-Fonctions

1. `charger_deputes_csv`
 - Description : Cette fonction charge les données des députés à partir d'un fichier CSV respectant le protocole défini par le programme.
 - Arguments :
 - `fichier` : Chaîne de caractères contenant le nom ou le chemin du fichier CSV.
 - `L` : Pointeur vers la structure contenant la liste dynamique des députés.
 - Retourne :
 - `true` si le chargement a réussi.
 - `false` en cas d'erreur (fichier introuvable ou format invalide).
 - Fonctionnalité : Lit le fichier CSV ligne par ligne, extrait les champs (identifiant, nom, parti, axes, patrimoine, votes), stocke les données en mémoire dynamique et trie la liste par identifiant afin de permettre une recherche rapide.
2. `trouver_depute_par_id`
 - Description : Recherche un député dans la liste à partir de son identifiant unique `mp_id`.
 - Arguments :
 - `L` : Liste des députés chargés en mémoire.
 - `mp_id` : Identifiant du député recherché.
 - Retourne :
 - L'indice du député dans la liste si trouvé.
 - `-1` si aucun député ne correspond à l'identifiant.
 - Fonctionnalité : Effectue une recherche dichotomique sur une liste préalablement triée afin d'optimiser le temps de recherche.

3. calculer_{axes_votes}

- Description : Calcule les axes idéologiques ■ réels ■ d'un député à partir de ses votes.
- Arguments :
 - **d** : Pointeur vers la structure représentant un député.
 - **eco** : Pointeur vers la valeur de sortie de l'axe économique.
 - **soc** : Pointeur vers la valeur de sortie de l'axe sociétal.
- Retourne : Rien.
- Fonctionnalité : Transforme les votes codés (Pour, Contre, Abstention, Absent) en valeurs numériques, applique des pondérations selon les thématiques, puis calcule des coordonnées sur les axes économique et sociétal.

4. calculer_{coherence}

- Description : Évalue le degré de cohérence entre les positions déclarées par un député et son comportement réel de vote.
- Arguments :
 - **d** : Pointeur vers la structure représentant un député.
- Retourne :
 - Un score de cohérence compris entre 0 et 100.
- Fonctionnalité : Calcule la distance entre les axes du programme et les axes issus des votes, puis transforme cette distance en un score de cohérence normalisé.

5. afficher_{fiche_complete}

- Description : Affiche toutes les informations détaillées concernant un député.
- Arguments :
 - **d** : Pointeur vers la structure représentant un député.
 - **log** : Pointeur vers un fichier de sortie (ou NULL pour affichage seul).
- Retourne : Rien.
- Fonctionnalité : Affiche l'identité du député, ses axes idéologiques, sa catégorie de patrimoine, la liste de ses votes, ses axes calculés et son score de cohérence, et peut également enregistrer ces informations dans un fichier.

6. afficher_{diagramme_2D}

- Description : Affiche une représentation graphique en deux dimensions des positions idéologiques d'un député.
- Arguments :
 - **d** : Pointeur vers la structure représentant un député.
 - **log** : Pointeur vers un fichier de sortie éventuel.
- Retourne : Rien.
- Fonctionnalité : Génère un diagramme ASCII positionnant le point correspondant au programme politique et celui issu des votes sur les axes économique et sociétal.

7. export_{profil}

- Description : Crée un fichier texte contenant le profil complet d'un député.
- Arguments :
 - **d** : Pointeur vers la structure représentant un député.
- Retourne : Rien.
- Fonctionnalité : Génère automatiquement un fichier horodaté contenant le pro-

tole explicatif, les informations du député et les résultats des calculs.

8. `export_comparaison`

- Description : Compare deux députés et exporte les résultats de la comparaison dans un fichier texte.
- Arguments :
 - `a` : Pointeur vers la structure représentant le premier député.
 - `b` : Pointeur vers la structure représentant le second député.
- Retourne : Rien.
- Fonctionnalité : Calcule la similarité des votes entre les deux députés, affiche un résumé de la comparaison en console, puis génère automatiquement un fichier texte horodaté de la forme `compare_<mpid1>_<mpid2>_<date>.txt`. Le fichier contient systématiquement le protocole explicatif, les informations d'identification des deux députés, le score de similarité des votes ainsi qu'un diagramme unique de comparaison positionnant **P1**, **V1**, **P2** et **V2** sur un même repère.

9. `afficher_top5_export`

- Description : Affiche et exporte le classement des députés les plus et les moins cohérents.
- Arguments :
 - `L` : Liste des députés chargés en mémoire.
- Retourne : Rien.
- Fonctionnalité : Calcule les scores de cohérence de tous les députés, trie les résultats et affiche les cinq meilleurs et les cinq moins cohérents, tout en générant un fichier de synthèse.

10. `afficher_diagramme_comparaison`

- Description : Affiche une représentation graphique ASCII en deux dimensions permettant de comparer visuellement deux députés sur un même repère idéologique.
- Arguments :
 - `log` : Pointeur vers un fichier de sortie (NULL si affichage en console uniquement).
 - `a` : Pointeur vers la structure représentant le député 1.
 - `b` : Pointeur vers la structure représentant le député 2.
- Retourne : Rien.
- Fonctionnalité : Calcule pour chaque député les axes idéologiques issus des votes, puis affiche un diagramme ASCII 2D unique contenant simultanément les quatre positions **P1** (programme député 1), **V1** (votes député 1), **P2** (programme député 2) et **V2** (votes député 2). Les superpositions éventuelles de marqueurs sur une même case de la grille sont détectées et signalées graphiquement afin d'assurer une lecture correcte du diagramme.

4.1.1 Fonctions utilitaires et techniques

En complément des fonctions principales décrites précédemment, le programme s'appuie sur plusieurs fonctions utilitaires et techniques destinées à améliorer la lisibilité du code, la robustesse de l'exécution et le confort d'utilisation en mode console.

Rôle général :

- Gestion de l’affichage en console (centrage du texte, séparateurs visuels, mise en page aérée).
- Nettoyage et validation des entrées utilisateur (gestion des erreurs de saisie, relecture, contrôle des valeurs).
- Fonctions de support pour les calculs intermédiaires (conversion des votes, normalisation des valeurs, calculs auxiliaires).
- Fonctions techniques internes liées au tri, à la recherche optimisée et à la génération de fichiers horodatés.

Retourne : Selon les cas, ces fonctions retournent une valeur simple (booléen, entier, flottant) ou ne retournent rien (`void`), leur objectif principal étant d’assister les fonctions principales.

Justification : Ces fonctions ne sont pas détaillées individuellement dans le rapport, car elles ne correspondent pas à des fonctionnalités visibles ou conceptuelles du programme. Leur rôle est essentiellement technique et vise à factoriser le code, éviter les répétitions et améliorer la clarté globale de l’implémentation, sans introduire de logique métier supplémentaire.

4.2 Programme principal (main)

1. Initialisation du programme :
 - Les structures de données nécessaires au stockage des députés sont initialisées.
 - Les variables de contrôle du programme (état du chargement, fichier CSV par défaut) sont mises en place.
2. Affichage du menu principal :
 - Un menu interactif est affiché à l’écran.
 - Il propose les différentes fonctionnalités du programme (chargement des données, affichage, comparaisons, classement, aide).
3. Chargement des données depuis un fichier CSV :
 - L’utilisateur est invité à fournir le nom d’un fichier CSV respectant le protocole défini.
 - La fonction `charger_deputes_csv` est appelée pour lire et stocker les données en mémoire dynamique.
 - En cas d’erreur (fichier invalide ou introuvable), l’utilisateur peut recommencer sans relancer le programme.
4. Sélection des fonctionnalités :
 - Tant que le fichier CSV n’est pas chargé, les autres fonctionnalités sont bloquées.
 - Une fois les données disponibles, l’utilisateur peut choisir parmi les différentes opérations proposées.

5. Affichage de la liste des députés :
 - La liste des députés chargés en mémoire est affichée à l'écran.
 - Chaque député est identifié de manière unique par son `mp_id`.
6. Affichage du profil complet d'un député :
 - L'utilisateur saisit l'identifiant `mp_id` du député recherché.
 - Le programme vérifie la validité de l'entrée et recherche le député correspondant.
 - Les informations détaillées sont affichées et automatiquement exportées dans un fichier texte.
7. Comparaison de deux députés :
 - L'utilisateur fournit les identifiants de deux députés.
 - Le programme calcule la similarité des votes et compare les scores de cohérence.
 - Les résultats sont affichés à l'écran et enregistrés dans un fichier de comparaison.
8. Classement des députés selon la cohérence :
 - Les scores de cohérence de tous les députés sont calculés.
 - Un classement des cinq députés les plus cohérents et des cinq moins cohérents est affiché.
 - Un fichier de synthèse est automatiquement généré.
9. Gestion des erreurs et navigation :
 - À chaque étape, les entrées utilisateur sont contrôlées.
 - En cas d'erreur, un message explicite est affiché et l'utilisateur peut recommencer l'opération en cours.
 - Le retour au menu principal est possible à tout moment.
10. Libération de la mémoire :
 - Avant la fin du programme, toute la mémoire allouée dynamiquement est correctement libérée.
11. Résumé du programme :
 - Le programme permet d'analyser la cohérence entre les positions affichées par des députés et leurs votes réels.
 - Il intègre la lecture de données externes, des calculs algorithmiques, des visualisations en console et des exports automatiques.
 - L'ensemble du traitement est structuré autour d'une interaction utilisateur simple et sécurisée.

5 Bibliothèques, Outils et technologies

Bibliothèques et outils utilisés :

stdio.h : Cette bibliothèque standard du langage **C** est utilisée pour toutes les opérations d'**entrée-sortie** du programme. Elle permet notamment l'affichage des menus, des profils et des diagrammes à l'écran à l'aide de **printf**, ainsi que la lecture des saisies

utilisateur et des fichiers **CSV** via des fonctions telles que **fgets** et **fprintf** pour l'écriture des fichiers de résultats.

stdlib.h : La bibliothèque **stdlib.h** fournit des fonctions essentielles au fonctionnement du programme, en particulier la **gestion de la mémoire dynamique** grâce à **malloc**, **realloc** et **free**, utilisées pour stocker un nombre variable de députés. Elle est également utilisée pour les **conversions numériques** (par exemple **atoi**, **strtol**) et certaines fonctions utilitaires liées à l'environnement d'exécution.

string.h : Cette bibliothèque standard en **C** est employée pour la manipulation des **chaînes de caractères**, notamment lors du traitement des lignes du fichier CSV. Elle permet de comparer, copier et nettoyer les chaînes de caractères correspondant aux noms des députés, aux partis politiques et aux différentes données textuelles du programme.

ctype.h : La bibliothèque **ctype.h** est utilisée pour analyser et nettoyer les entrées utilisateur, en particulier pour détecter et ignorer certains caractères non souhaités (espaces, fins de ligne). Elle contribue à rendre la saisie plus robuste face aux erreurs de l'utilisateur.

math.h : Cette bibliothèque est utilisée pour effectuer les **calculs mathématiques** nécessaires à l'évaluation de la cohérence politique. Elle permet notamment le calcul de distances entre les axes idéologiques à l'aide de fonctions comme **sqrt**, afin d'obtenir un score de cohérence compris entre **0** et **100**.

time.h : Cette bibliothèque standard est utilisée pour manipuler le **temps système**. Dans ce projet, elle sert à générer un **horodatage** lors de la création des fichiers de sortie (profils, comparaisons, classement Top 5), afin de produire des noms de fichiers uniques et d'éviter d'écraser des résultats précédents (fonctions comme **time**, **localtime** et **strftime**).

stdarg.h : La bibliothèque **stdarg.h** permet de gérer des fonctions à **nombre variable d'arguments**. Elle est utilisée ici pour implémenter une fonction d'affichage centrée de type "printf" (ex : **cprintf_center**), qui accepte différents formats et paramètres. Elle repose sur l'utilisation de mécanismes tels que **va_list**, **va_start** et **vsprintf**.

Outils et technologies utilisés :

Code : :Blocks : Un environnement de développement intégré (**IDE**) gratuit dédié principalement aux langages **C** et **C++**. Il a été utilisé dans ce projet pour écrire le code source, compiler le programme et exécuter les tests. Ses fonctionnalités, telles que la **coloration syntaxique**, la gestion des erreurs de compilation et le débogage intégré, ont facilité le développement et la correction du programme.

ChatGPT (OpenAI) : Un outil basé sur l'**intelligence artificielle**, développé par **OpenAI**, capable d'analyser des consignes et de fournir des réponses textuelles structurées.

rées. Dans ce projet, il a été utilisé comme **outil d'assistance pédagogique**, notamment pour clarifier certains concepts, structurer le programme, améliorer la lisibilité du code et aider à la résolution de problèmes de programmation en langage C.

LibreOffice Calc : Un tableur libre utilisé pour créer, organiser et modifier les fichiers **CSV** servant de données d'entrée au programme. Il a permis de structurer correctement les colonnes, de vérifier la cohérence des valeurs (axes, votes, patrimoine) et d'exporter les données dans un format compatible avec la lecture en langage C.

Bloc-notes (Notepad) : Utilisé pour ouvrir et vérifier les fichiers texte générés automatiquement par le programme (profils complets, comparaisons et classements). Cet outil simple a permis de contrôler la lisibilité des fichiers de sortie et de s'assurer du respect du protocole défini.

6 Travail réalisé

Au cours de la réalisation de ce projet, nous avons travaillé **en binôme**, ce qui nous a permis de confronter nos idées, de discuter des choix de conception et de répartir les tâches de manière efficace. Ce travail collaboratif a favorisé une meilleure organisation du projet et une prise de recul sur les différentes solutions envisagées.

Face à la complexité du sujet, nous avons adopté une **approche méthodique et progressive**, consistant à découper le projet en plusieurs **étapes fonctionnelles clairement identifiées**. Le développement s'est articulé autour de modules distincts : lecture et validation des données depuis un fichier CSV, gestion dynamique des députés en mémoire, calcul des axes idéologiques à partir des votes, évaluation d'un score de cohérence, affichage structuré en console et génération automatique de fichiers de résultats.

Une part importante du travail a porté sur la **conception d'un protocole de données précis et reproductible**. Plutôt que d'intégrer des données directement dans le code, nous avons défini un format CSV strict (colonnes, types, codification des votes, axes idéologiques, patrimoine), permettant à un utilisateur externe de produire ses propres fichiers. Dans ce cadre, l'utilisateur peut s'appuyer sur des **outils d'intelligence artificielle tels que ChatGPT** pour générer un fichier CSV à partir d'informations collectées, à condition de respecter rigoureusement le protocole défini par le programme. Un travail de **vérification humaine** reste néanmoins indispensable pour assurer la cohérence et la validité des données.

Le programme a également été conçu pour conserver une **trace écrite systématique des résultats**. Lors de l'affichage d'un profil complet, d'une comparaison entre deux députés ou d'un classement des plus et moins cohérents, un fichier texte est automatiquement généré. Ces fichiers contiennent le protocole explicatif ainsi que les résultats obtenus, ce qui facilite l'analyse, la relecture et l'exploitation ultérieure des données.

Enfin, une attention particulière a été portée à la **gestion des erreurs et à l'expérience utilisateur**. Des contrôles ont été intégrés à chaque étape de saisie afin d'éviter les entrées invalides, avec des messages clairs et la possibilité de recommencer sans re-

lancer entièrement le programme. Cette organisation permet une navigation plus fluide, proche d'un fonctionnement par "pages", malgré les contraintes d'un affichage en mode console.

7 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours de ce projet. L'une des principales concerne la **gestion de la mémoire dynamique en langage C**, notamment lors du stockage d'un nombre variable de députés et du redimensionnement des structures de données. Cette étape a nécessité une compréhension précise du fonctionnement de l'allocation et de la libération de la mémoire afin d'éviter des erreurs telles que les fuites mémoire ou les accès invalides.

Une autre difficulté importante a porté sur le **traitement des entrées utilisateur et des fichiers CSV**. Il a fallu gérer les erreurs de saisie, les formats incorrects et les problèmes d'affichage liés aux caractères spéciaux et aux accents. Ces contraintes ont conduit à imposer un protocole strict (utilisation d'un identifiant unique `mp_id`, absence d'accents) et à renforcer les mécanismes de validation dans le programme.

La conception du **diagramme 2D en console** a également constitué un défi. Les limitations de l'affichage textuel ont nécessité plusieurs ajustements afin d'obtenir une représentation lisible et compréhensible, tout en maintenant une correspondance claire entre les axes déclarés (programme) et les axes calculés à partir des votes.

Enfin, certaines améliorations envisagées, comme une interface graphique complète ou une visualisation plus avancée des données, n'ont pas pu être mises en œuvre dans le temps imparti, en raison des contraintes techniques et du cadre pédagogique du projet.

8 Bilan

8.1 Conclusion

Ce projet nous a permis de développer un programme complet en **langage C**, mobilisant de nombreuses notions fondamentales vues en L2, telles que les **structures de données**, la **mémoire dynamique**, la **lecture et l'écriture de fichiers**, la **recherche optimisée** et la structuration modulaire du code.

L'analyse de la cohérence entre les positions affichées par des députés et leur comportement de vote constitue un cas d'étude concret, combinant une problématique actuelle avec des traitements algorithmiques accessibles mais non triviaux. Le projet met également en évidence l'importance d'un **protocole de données clair et rigoureux**, indispensable pour exploiter correctement des données externes.

Ce travail en binôme nous a permis de renforcer à la fois nos compétences techniques et notre capacité à collaborer autour d'un projet de programmation structuré, tout en développant une approche plus méthodique de la conception logicielle.

8.2 Perspectives

Plusieurs perspectives d'amélioration pourraient être envisagées pour prolonger ce projet. Tout d'abord, une évolution naturelle serait la mise en place d'une **interface graphique réelle**, permettant une visualisation plus intuitive des diagrammes et des résultats, et améliorant significativement l'expérience utilisateur.

Il serait également possible d'enrichir le modèle d'analyse en intégrant un nombre plus important de votes, des pondérations plus fines ou des comparaisons temporelles, afin d'étudier l'évolution de la cohérence d'un député au fil du temps.

Enfin, une perspective intéressante consisterait à automatiser davantage la **collecte et la vérification des données**, en combinant le programme avec des outils externes d'aide à la génération de CSV, tout en conservant un contrôle humain sur la qualité et la fiabilité des informations.

9 Bibliographie

[label] Auteur, TITRE, editeur, annee

[LAM94] L. LAMPORT, *TEX : A Document preparation system*, Addison-Wesley, 1994

10 Webographie

[CAT] savoircoder.fr/cat

<https://koor.fr/C/cstring/cstring.wp>

[https://fr.wikipedia.org/wiki/Coh%C3%A9rence_\(donn%C3%A9es\)](https://fr.wikipedia.org/wiki/Coh%C3%A9rence_(donn%C3%A9es))

<https://fr.wikipedia.org/wiki/D%C3%9Cmagogie>

https://fr.wikipedia.org/wiki/Parti_politique

<https://github.com/HowTheyVote/data>

[https://www.europarl.europa.eu/topics/en/article/20240513ST021505/
how-to-follow-the-activities-and-votes-of-your-mep](https://www.europarl.europa.eu/topics/en/article/20240513ST021505/how-to-follow-the-activities-and-votes-of-your-mep)

<https://howtheyvote.eu/>

<https://www.france24.com/en/europe/20240530-french-far-right-s-rhetoric-at-odds-wi>

11 Annexes

Annexe A : Cahier des charges

Objectif du projet :

Développer un programme en langage C permettant d'analyser la **cohérence politique de députés** en comparant leurs **positions déclarées** (axes idéologiques) à leurs **votes réels** sur plusieurs scrutins, à partir de données structurées dans un fichier CSV.

Fonctionnalités attendues :

1. *Préparation des données :*

- Chargement d'un fichier CSV respectant un **protocole strict** (colonnes, types de données, codification des votes).
- Identification unique des députés à l'aide d'un **identifiant** `mp_id` afin d'éviter toute ambiguïté.
- Vérification de la validité globale du fichier (format, valeurs attendues).

2. *Traitement des informations politiques :*

- Lecture des axes idéologiques déclarés (**économique** et **sociétal**).
- Interprétation des votes codés (**Pour**, **Contre**, **Abstention**, **Absent**) sur plusieurs thématiques.
- Calcul automatique des **axes idéologiques issus des votes**.

3. *Analyse de la cohérence :*

- Calcul d'un **score de cohérence** compris entre 0 et 100 pour chaque député.
- Possibilité de comparer deux députés selon leurs votes et leurs axes idéologiques.
- Établissement d'un **classement** des députés les plus et les moins cohérents.

4. *Affichage et export des résultats :*

- Affichage des profils détaillés en console, incluant les informations d'identité, les axes idéologiques, les votes et le score de cohérence.
- Représentation graphique simplifiée à l'aide d'un **diagramme 2D ASCII** positionnant les axes économique et sociétal.
- Lors d'une comparaison entre deux députés, affichage d'un **diagramme unique de comparaison** contenant simultanément les marqueurs :
 - **P1** : position **programme** (axes issus du fichier CSV) du député 1,
 - **V1** : position **votes** (axes calculés à partir des scrutins) du député 1,
 - **P2** : position **programme** du député 2,
 - **V2** : position **votes** du député 2.
- En cas de superposition de plusieurs marqueurs sur une même case du diagramme, le symbole ****** est utilisé afin de signaler cette collision visuelle.
- Génération automatique de **fichiers texte** (profil, comparaison, classement) enregistrés dans le dossier du programme et incluant systématiquement le protocole explicatif ainsi que les résultats affichés.

Exigences techniques

Langage : Le programme doit être écrit en **langage C**, avec une structuration claire du code et des commentaires explicatifs.

Interface utilisateur : Le programme fonctionne en **ligne de commande**, avec un menu interactif, des messages explicites et une gestion des erreurs permettant à l'utilisateur de recommencer une action.

Portabilité : Le programme doit pouvoir être compilé et exécuté sur différents systèmes (Windows, Linux), sans dépendance à des bibliothèques graphiques externes.

Fiabilité : Le programme doit gérer correctement les erreurs de saisie et les fichiers invalides, sans provoquer d'arrêt brutal.

Livraison

Code source : Le code source complet du programme doit être fourni, commenté et structuré.

Documentation : Une documentation expliquant le fonctionnement du programme, le protocole CSV et les méthodes de calcul utilisées doit être fournie.

Tests : Des tests doivent être réalisés avec différents fichiers CSV afin de vérifier la robustesse et la cohérence des résultats.

Annexe B : Manuel utilisateur

1. Démarrage du programme

Lancer le programme depuis la ligne de commande. Un menu principal s'affiche, proposant les différentes fonctionnalités disponibles.

2. Chargement des données

L'utilisateur doit charger un **fichier CSV** respectant le protocole du programme. Ce fichier peut être créé manuellement ou généré à l'aide d'un outil externe (par exemple ChatGPT), à condition de respecter strictement la structure imposée.

3. Consultation de la liste des députés

Une fois les données chargées, l'utilisateur peut afficher la liste complète des députés disponibles. Chaque député est identifié par son **mp_id**, utilisé dans les opérations suivantes.

4. Affichage du profil d'un député

L'utilisateur saisit l'identifiant **mp_id** d'un député. Le programme affiche alors :

- Les informations générales du député.
- Les axes idéologiques déclarés.
- Les axes calculés à partir des votes.
- Le score de cohérence.
- Un diagramme 2D ASCII.

Un fichier texte récapitulatif est automatiquement généré dans le dossier du programme.

5. Comparaison de deux députés :

- L'utilisateur saisit deux identifiants **mp_id** (0 permet d'annuler).
- Le programme affiche un résumé des deux députés (nom, parti, mp_id), puis une **similarité de votes** (en %).
- Ensuite, le programme affiche un **diagramme unique de comparaison** (axes économie/sociétal) contenant les 4 positions suivantes :
 - **P1** : position **programme** du député 1
 - **V1** : position **votes** du député 1 (calculée)
 - **P2** : position **programme** du député 2
 - **V2** : position **votes** du député 2 (calculée)
- Si plusieurs points se superposent sur une même case, le programme affiche ******.

6. Classement des députés

Le programme peut afficher un **Top 5 des députés les plus cohérents** et un **Top 5 des moins cohérents**. Un fichier de synthèse est automatiquement créé.

7. Gestion des erreurs et navigation

En cas de saisie invalide, le programme affiche un message d'erreur et propose de recommencer l'opération sans quitter le programme.

8. Fin du programme

Lorsque l'utilisateur quitte le programme, toute la mémoire allouée dynamiquement est correctement libérée afin d'éviter les fuites de mémoire.

Notes supplémentaires

Il est recommandé d'éviter les accents et caractères spéciaux dans les fichiers CSV afin de garantir un affichage correct en console. Le programme repose sur une gestion dynamique de la mémoire et assure une traçabilité des résultats via les fichiers générés automatiquement.

Annexe C : Exemple d'exécution du projet

Cette annexe présente un exemple complet d'exécution du programme, illustrant les principales fonctionnalités développées dans le cadre du projet. Les captures d'écran suivantes montrent le déroulement du programme depuis son lancement jusqu'à l'analyse des résultats, en passant par le chargement des données, l'affichage des profils, la comparaison et le classement des députés.

```
=====
                        MENU
=====

      0. Aide / protocole
      1. Charger CSV

      2. Afficher liste des deutes
      3. Afficher profil complet (mp_id)
      4. Comparer deux deutes (mp_id, mp_id)

      5. Top 5 plus coherents / moins coherents

      9. Quitter

-----
      Choix :
```

FIGURE 1 – Lancement du programme et affichage du menu principal. Le programme démarre en mode console et propose un menu interactif permettant à l'utilisateur de choisir les différentes opérations disponibles.

```
=====
                        CHARGEMENT CSV
=====

Conseil : place le CSV dans le meme dossier que le programme.

      Nom du fichier CSV (Entree pour 'deutes.csv') :

      Succes : 20 deutes charges depuis 'deutes.csv'.
      Les deutes ont ete tries par mp_id (recherche rapide).

      Appuyez sur Entree pour revenir au menu...
```

FIGURE 2 – Chargement des données depuis un fichier CSV conforme au protocole du programme. Les informations des députés sont lues et stockées en mémoire dynamique.

LISTE DES DEPUTES (20)		
mp_id	Nom	Parti
1	Camille Bernard	RE
2	Hugo Legrand	RN
3	Ines Morel	LFI
4	Yanis Dubois	LR
5	Sarah Nguyen	PS
6	Nolan Petit	EELV
7	Maya Diallo	MoDem
8	Arthur Cohen	Horizons
9	Lina Garcia	RN
10	Mehdi Benali	LFI
11	Chloe Martin	RE
12	Baptiste Rousseau	LR
13	Aicha Traore	PS
14	Noe Lefevre	EELV
15	Emma Fournier	RE
16	Lucas Girard	RN
17	Sofia Lopez	MoDem
18	Rayan Haddad	Horizons
19	Manon Perrin	LR
20	Jules Lambert	PS
Appuyez sur Entree pour revenir au menu...		

FIGURE 3 – Affichage de la liste des députés chargés en mémoire. Chaque député est identifié par un identifiant unique (mp_id), utilisé pour les opérations suivantes.

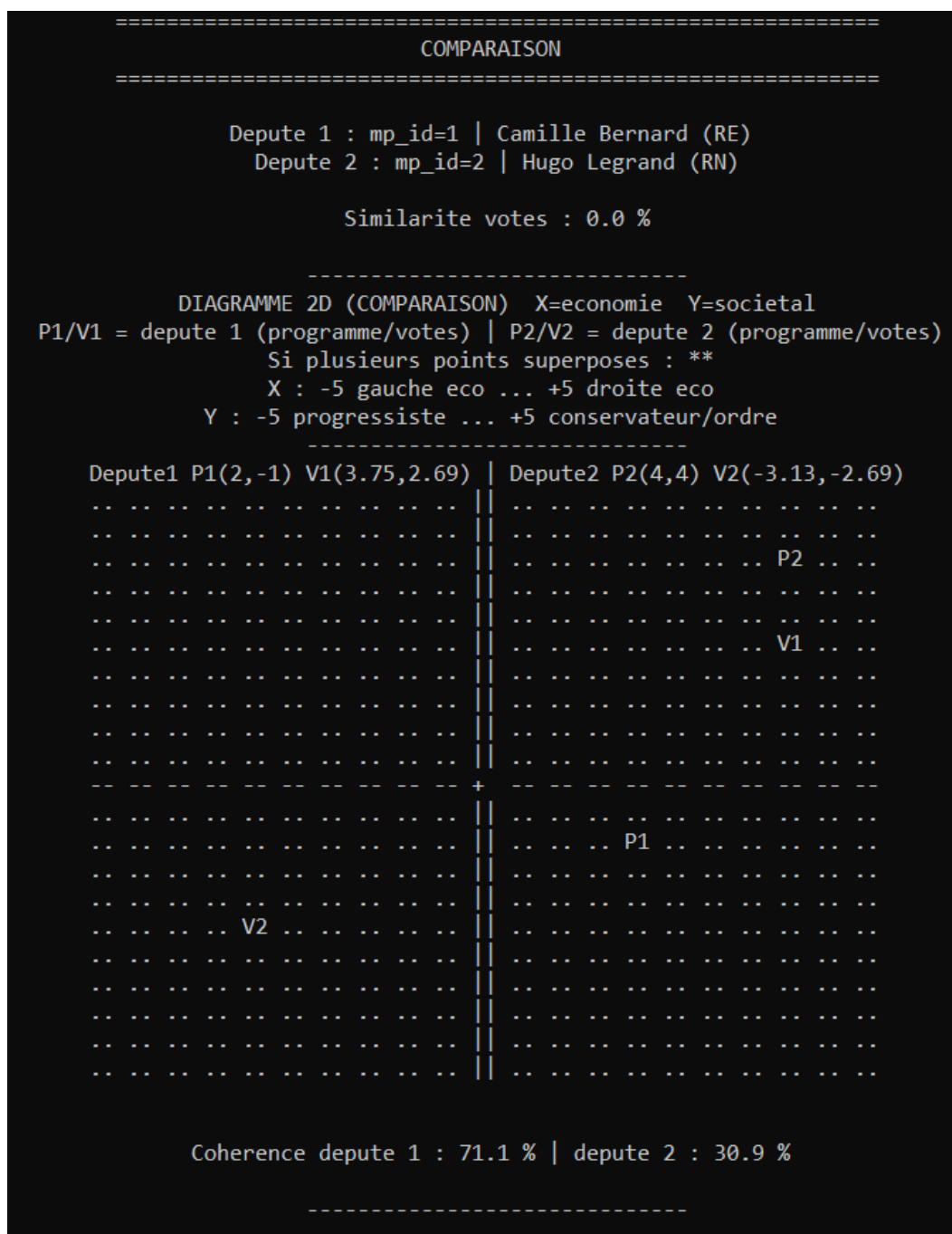


FIGURE 5 – Comparaison de deux députés avec un diagramme unique affichant P1/V1 et P2/V2 sélectionnés par leur `mp_id`. Les différences de votes, d’axes idéologiques et de cohérence sont affichées et analysées.

```
=====
CLASSEMENT COHERENCE
=====

Top 5 : plus cohérents
-----
#1 mp_id=8 | Arthur Cohen | 89.3 %
#2 mp_id=18 | Rayan Haddad | 89.3 %
#3 mp_id=13 | Aicha Traore | 75.0 %
#4 mp_id=12 | Baptiste Rousseau | 72.2 %
#5 mp_id=19 | Manon Perrin | 71.6 %

Top 5 : moins cohérents
-----
#1 mp_id=9 | Lina Garcia | 30.8 %
#2 mp_id=2 | Hugo Legrand | 30.9 %
#3 mp_id=16 | Lucas Girard | 32.6 %
#4 mp_id=14 | Noe Lefevre | 51.6 %
#5 mp_id=3 | Ines Morel | 54.1 %

=====

Fichier enregistré : top5_20260102_011641.txt

Appuyez sur Entree pour revenir au menu...
```

FIGURE 6 – Classement des députés selon leur score de cohérence. Le programme affiche les cinq députés les plus cohérents et les cinq moins cohérents, avec les scores correspondants.

```
=====
                                MENU
=====

      0. Aide / protocole
      1. Charger CSV

      2. Afficher liste des deutes
      3. Afficher profil complet (mp_id)
      4. Comparer deux deutes (mp_id, mp_id)

      5. Top 5 plus coherents / moins coherents

      9. Quitter

      -----
      Choix : gd

Mauvaise entree : choisissez un numero du menu.
Conseil : tapez 0 pour afficher l'aide.

Appuyez sur Entree pour continuer...
```

FIGURE 7 – Exemple de gestion des erreurs de saisie. En cas d'entrée invalide, un message explicite est affiché et l'utilisateur peut recommencer l'opération sans quitter le programme.

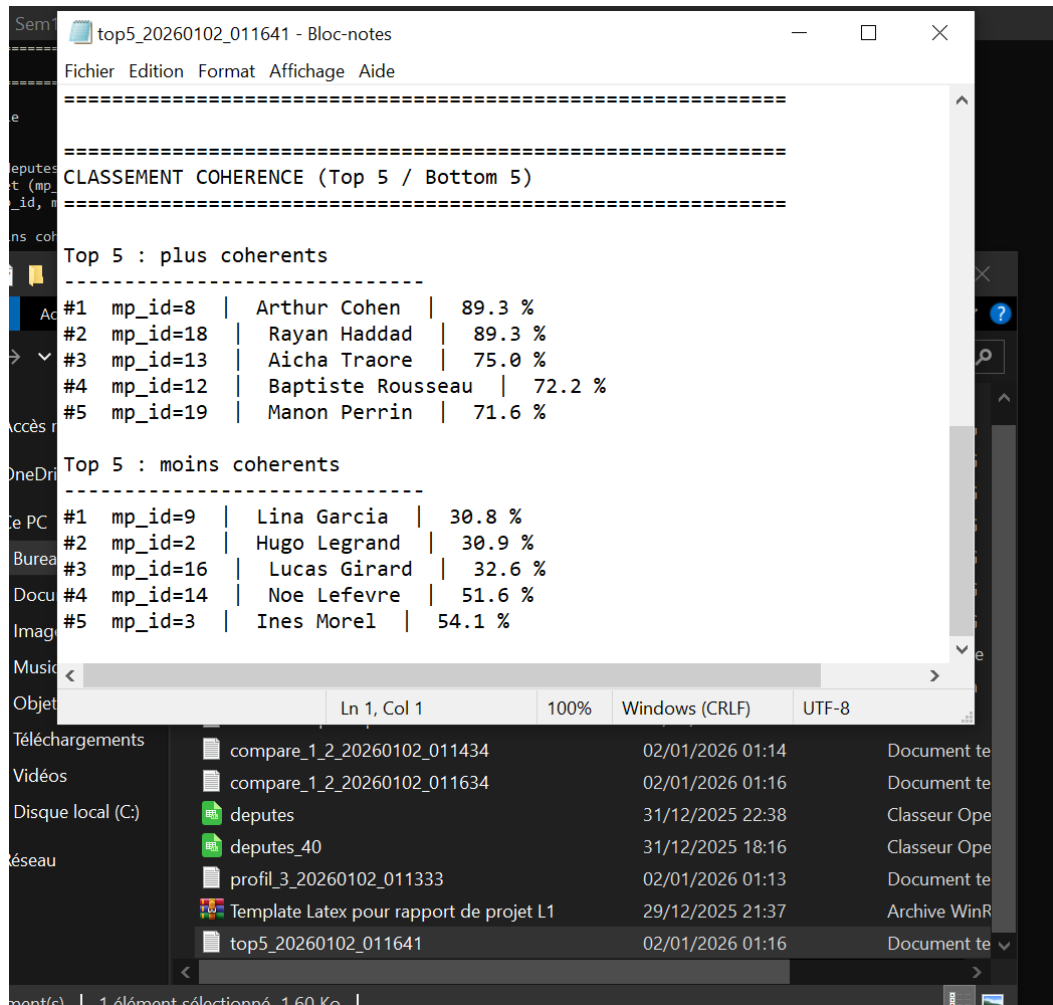


FIGURE 8 – Fichiers texte générés automatiquement par le programme. Ces fichiers permettent de conserver une trace des profils, comparaisons et classements réalisés.